붙임1: Novice Project 계획서 서식 샘플+

Project 계획서

프로젝트명	**국문**	데이터 통신 팀 프로젝트		
	영문	Data Communication Team Project		
학부(과)	컴퓨터공학과		**담당교수**	임환섭

프로젝트 수행팀	**팀명**	12팀		
	학번	**성명**	**연락처(휴대폰)**	**E-mail**
	20222963	나승주	010-8806-1684	naseungju02@gmail.com
	20222370	신승준	010-9867-3655	tmdwns7655@naver.com
	20223520	정희영	010-2650-8492	hy5280@daum.net
	20221963	허재민	010-3302-6706	hur6706@gmail.com

프로젝트수행기간	2026. 09. 22 - 2026. 12. 21 (2개월)

동의대학교 컴퓨터공학과 관계규정과 지시사항을 준수하면서 데이터 통신 팀 프로젝트를 성실히 수행하고자 하며, 이에 아래와 같이 프로젝트 계획서를 제출합니다.

2026년 09월 23일

프로젝트 팀장 : 나승주

담당(지도)교수 : 임환섭

동의대학교 컴퓨터공학 학부(과)장 귀하

〈 요 약 문 〉

<table>
<tr><td>프로젝트 목표</td><td>본 프로젝트는 실시간 네트워크 대역폭 변동에 맞춰 비디오 화질과 버퍼 상태를 동적으로 제어하는 Python 기반 적응형 비디오 스트리밍(ABR) 시뮬레이터 개발을 목표로 한다.

3대 핵심 원칙인 ①실행가능성(로컬 완결형 구동), ②시연 가능성(실시간 라이브 데모), ③보고서정합성(이론 수식과 실측 데이터 일치)을 충족한다. 대역폭 급락 및 패킷 손실 환경에서 ABR 제어 알고리즘이 사용자 체감 품질(QoE)을 보장하고 버퍼 끊김을 방지하는 동작 구조를 소프트웨어로 증명한다.</td></tr>
<tr><td>내용</td><td>본 시뮬레이터는 네트워크 환경 모사, ABR 제어, UI 시각화, 로그 수집의 4개 핵심 모듈로 구성된다.

1. 가상 채널 모사 (network_channel.py): 대역폭, 패킷 손실률, RTT 지연, Jitter 수식을 반영하여 실시간 유효 처리량을 연산한다.

2. ABR 제어 엔진 (abr_engine.py): 잔여 버퍼 시간과 유효 처리량을 기반으로 최적 비트레이트(240p ~ 1080p)를 선택하고, Playing/Rebuffering 버퍼 상태 머신을 제어한다.

3. 대시보드 UI (app_ui.py): Streamlit 기반으로 실시간 네트워크 환경 조절 슬라이더 및 비트레이트/버퍼 수준 변동 그래프를 동적 렌더링한다.

4. 이벤트 로깅 (logger.py): 세그먼트 요청/응답 및 화질 변경 이벤트를 CSV로 기록한다.

팀원 4명이 백엔드, 통신 수식, UI, 시연 테스트로 역할을 분담하고 GitHub 커밋 이력으로 기여도를 가시화하며, 13주차 알파버전을 거쳐 15주차에 필수 5종 산출물(프로그램, 소스코드, 매뉴얼, 시나리오, 보고서)을 최종 제출한다.</td></tr>
<tr><td>발표 및 시연 계획</td><td>14주차 최종 발표회(15분 배정)의 33%(5분)를 실시간 라이브 데모에 배정한다. python main.py --mode demo 구동 후 대역폭 급락(10Mbps ---> 1Mbps) 주입 시, 비트레이트 하향 조정을 통해 버퍼 끊김이 방지되는 과정을 실시간 그래프로 시연한다. 현장 오류 대비 3~5분 OBS 사전 녹화 백업 영상과 예외 처리 가드를 사전 반영하여 시연 안정성을 확보한다.</td></tr>
<tr><td>Keywords</td><td>적응형 비디오 스트리밍

</td><td>흐름 및 혼잡 제어

</td><td>사용자 체감 품질

</td></tr>
</table>

목 차

Ⅰ. 서론 ····· #

1. 프로젝트의 배경 ····· #

2. 프로젝트의 목표 ····· #

Ⅱ. 프로젝트 수행 내용 ····· #

1. 프로젝트의 내용 ····· #

2. 프로젝트 추진전략 및 방법(체계) ····· #

Ⅲ. 팀 발표 및 실시간 시연 계획 ····· #

1. 프로젝트 발표 구성안 ····· #

2. 시연 리스크 관리 전략 ····· #

Ⅳ. 기타 ····· #

I. 서론

1. 프로젝트의 배경

일상적 호기심에서 시작된 기술적 접근: 유튜브나 넷플릭스 시청 중 Wi-Fi 신호가 불안정해질 때 영상이 뚝 끊기는 대신 '갑자기 화질만 낮아지며 끊김 없이 재생이 유지되는 현상'은 누구나 한 번쯤 겪어본 일상적인 경험이다.

적응형 비디오 스트리밍(ABR)의 기술적 보편화: 이러한 현상의 이면에는 실시간 네트워크 대역폭 변동을 인지하여 비디오 화질(Bitrate)과 버퍼 상태를 동적으로 제어하는 적응형 비디오 스트리밍(Adaptive Bitrate Streaming, ABR) 기술이 자리 잡고 있다.

2. 프로젝트의 목표

가. 공학적 최종목표 (정량적 기능 및 스펙)

(1) 실시간 가상 네트워크 채널 모사 모듈 구축:

1. 대역폭(0.5 Mbps ~ 20.0 Mbps) 및 지연/패킷 손실률 동적 가변 슬라이더 조절 기능 구현.

2. RTT 지연(20ms ~ 500ms) 및 패킷 손실률(0% ~ 20%) 반영 유효 처리량 연산 수식 적용.

(2) 흐름 및 혼잡 제어 기반 ABR 제어 엔진 개발:

1. 잔여 버퍼 레벨(Target Buffer: 10초~30초) 및 유효 대역폭 판단 기반 4단계 화질(240p/480p/720p/1080p) 자동 전환 알고리즘 완성.

(3) 실시간 시각화 대시보드 UI 및 성능 비교 그래프 구축:

1. Streamlit 활용, 네트워크 변동 발생 시 1초 이내 비트레이트 및 버퍼 변화 동적 렌더링.

나. 비공학적 최종목표

(1) 통신 메커니즘의 직관적 증명:

복잡한 통신 프로토콜의 대역폭 조절 및 혼잡 제어 원리를 '화질 변동'과 '버퍼 그래프'라는 직관적 시각 요소로 바꾸어 통신 이론의 전달력을 극대화한다.

II. 프로젝트 수행내용

1. 프로젝트의 내용

1) 목표 달성을 위한 프로젝트의 구체적인 내용

(1) 공학적 고려요소 및 현실적 제한조건:

가. 공학적 요소 (설계/제작/시험/평가):

1. 가상 채널 모사 (src/network_channel.py):

실시간 대역폭 변동, 패킷 손실률, RTT 지연 수식을 적용하여 유효 처리량을 계산하는 모듈 구현.

2. ABR 및 제어 엔진 (src/abr_engine.py):

잔여 버퍼 상태 머신(PLAYING, REBUFFERING, DROP)과 대역폭 변동을 인지하여 비트레이트를 동적으로 결정하는 Rule-based 알고리즘 개발.

3. 대시보드 UI (src/app_ui.py):

Streamlit 기반으로 실시간 대역폭 조절 슬라이더 및 비트레이트/버퍼 수준 동적 그래프 렌더링.

4. 로그 모듈 (src/logger.py):

세그먼트 요청/응답 및 화질 전환 이벤트를 실시간으로 기록.

5. 현실적 제한조건 (안정성/신뢰성/실행가능성):

안정성/신뢰성 (오류 대응 가드): 대역폭 슬라이더 0Mbps 세팅, 패킷 손실률 100% 등 극한값 입력 시 프로그램 다운을 방지하는 입력 예외 처리 가드 사전 구축.

6. 실행 가능성 및 독립성:

외부 네트워크 환경 변화에 영향을 받지 않는 100% 로컬 실행 구조 구축 및 python main.py --mode demo 명령어를 통한 직관적인 라이브 시연환경 제공.

나. 비공학적 고려요소 (연구방법/프로젝트 관리):

1. GitHub 기반 협업 및 기여도 증명:

GitHub Private 저장소(data-comm-abr-project)를 개설하고 팀원별 브랜치 분기 및 Pull Request(PR) 검토 후 병합 방식을 통해 개인별 기여도(커밋 이력)를 가시화함.

2. 5종 산출물 체계적 관리:

15주차 일괄 제출을 위해 실행 프로그램, 소스코드, 실행 매뉴얼(README.md), 시연 시나리오(OBS 백업 영상 포함), 최종 보고서(AI 활용 기록 부록 포함)를 단계별로 완비함.

2) 프로젝트 설계(or 내용)의 독창성:

단순 이론적 조사에 그치지 않고, 네트워크 지연 및 패킷 손실 시 통신 프로토콜이 대역폭을 동적으로 조절(흐름/혼잡 제어)하는 원리를 사용자가 직접 슬라이더로 조절하며 실시간 동적 그래프로 비교·증명할 수 있는 시뮬레이터 형태로 직접 구현함.

3) 기타:

14주차 최종 발표회 33% 실시간 라이브 데모 배정: 발표 시간 15분 중 5분을 시연에 배정하며, 현장 시연 실패에 대비한 OBS 3~5분 백업 영상 사전 녹화 준비 병행.

2). 프로젝트 추진전략 및 방법(체계)

1) 프로젝트 추진 일정 및 방법

(1) 정보수집, 목표 달성과 문제점 해결을 위한 추진 일정 및 적용 방법:

정보수집 및 연구:

데이터통신 교재 기반 네트워크 수식 분석, ABR 알고리즘 논문 및 자료 조사.

모듈별 독립 개발 및 1차 병합:

4~8주차 동안 백엔드(ABR/채널), 프론트엔드(UI), 테스터 모듈을 분리 개발 후 PR 코드 병합.

마일스톤 달성:

핵심 기능 70% 이상 작동되는 알파버전 라이브 시연 수행.

최종 시연 및 일괄 제출:

14주차 33% 실시간 라이브 데모 진행 및 15주차 필수 5종 제출 산출물 완비.

[표 1] 프로젝트 추진 일정

No	프로젝트 활동 내용	추진 일정															기간 (주)	추진 방법
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15		
1	환경 구축 및 규격 정리																2	- GitHub(team-project)개설 - CLI(--mode demo) 및 브랜치 세팅 - 4주차 수행계획서 작성 및 제출
2	모듈별 핵심 기능 개발																1	- Rule-based ABR 알고리즘 작성 - 네트워크 대역폭/손실 모사 개발 - UI 슬라이더 및 로그 모듈 개발
3	알고리즘 고도화 & 예외 처리																1	- 버퍼 상태 전환 제어 로직 작성 - 극한값 입력 시 다운 방지 가드 작성 - 검증 시나리오 단위 테스트 케이스 작성
4	1차 코드 병합 및 연동 테스트																1	- main 브랜치 코드 병합 및 연동 확인 - requirements.txt 작성 - 통합 테스트 및 버그 트래킹
5	시연 모드 구현 및 알파버전 시연																5	- python main.py --mode demo 파이프라인 구축 - 13주차 마일스톤: 핵심 기능 70% 구동 알파버전 라이브 시연
6	최종 라이브 데모 & 5종 산출물 제출																2	- 14주차 15분 발표회 (실시간 라이브 시연 + OBS 백업 영상 완비) - 15주차 필수 5종 제출물 일괄 제출

2) 프로젝트 추진 체계

- 팀 구성원의 역할 분담 및 수행내용 상세 기술:

팀장 나승주 (팀장 / 백엔드):

ABR 알고리즘 제어 로직, 버퍼 상태 머신 모듈 개발 및 GitHub 저장소·시연 플래그 구조 총괄.

팀원 허재민 (통신 / 보고서):

가상 네트워크 채널 모사, 다운로드 시간/유효 처리량 수식 연산 및 최종 보고서 작성 책임.

팀원 신승준 (UI / 매뉴얼):

Streamlit 대시보드 UI, 실시간 동적 그래프 렌더링 및 실행 매뉴얼 (README.md, requirements.txt) 작성.

팀원 정희영 (테스터 / 시연):

시스템 모니터링 로그 모듈, 3가지 통합 테스트, 실시간 시연 시나리오 가이드라인 작성 및 OBS 백업 영상 제작.

[표 2] 프로젝트 추진체계

성명 / 학번	수행내용	역할
나승주	■ ABR 알고리즘 개발: 대역폭/버퍼 상태 기반 비트레이트 동적 결정 로직 구현 ■ 버퍼 상태 머신 구축: 재생, 버퍼링, 드롭 상태 전환 제어 모듈 작성 ■ 저장소 및 시연 관리: GitHub 브랜치 관리 및 시연용 CLI(--mode demo) 환경 구축	팀장
20222963		
허재민	■ 가상 채널 개발: 대역폭 변동, 패킷 손실, 지연 모사 시뮬레이션 채널 구축 ■ 수식 연산 처리: 다운로드 시간 및 유효 처리량(Throughput) 수식 계산 모듈 작성	팀원
20221963		
신승준	■ 대시보드 UI 구축: Streamlit 기반 네트워크 조건(대역폭/손실률) 조절 슬라이더 구현 ■ 실시간 시각화: 화질 변동, 버퍼 잔여량, 비트레이트 변화 동적 그래프 렌더링 ■ 실행 매뉴얼 작성: README.md 및 의존성 명세(requirements.txt) 작성	팀원
20222370		
정희영	■ 모니터링 & 로그 모듈: 비디오 요청/응답 및 화질 전환 이벤트 실시간 로그 구현 ■ 통합 테스트 수행: 네트워크 급변 시나리오 검증 및 버그 트래킹 ■ 시연 준비: 실시간 시연 시나리오 가이드라인 수립 및 OBS 백업 영상 사전 녹화	팀원
20223520		

III. 팀 발표 및 실시간 시연 계획

1. 프로젝트 발표 구성안

01. 팀 소개 및 문제 정의 3분

발표 내용: 팀원별 역할 소개 및 연구 배경/목적 설명

핵심 내용: 대역폭 변동 및 패킷 손실 환경에서 ABR 알고리즘 및 버퍼 관리의 필요성 제시

02. 시스템 구조 및 스택 2분

발표 내용: 전체 시스템 아키텍처 및 모듈 간 데이터 흐름 설명

핵심 내용: ABR 엔진, 가상 채널 시뮬레이터, Streamlit UI, 모니터링 로그 모듈 연동 구조 및 기술 스택 명세

03. 실시간 프로그램 시연 5분(발표의 33%)

시연 방식: 사전 녹화 영상이 아닌 python main.py --mode demo 명령어를 통한 Live 시연

시연 시나리오: 실시간 네트워크 조건(대역폭, 손실률) 변경에 따른 비트레이트 동적 전환, 버퍼 상태 변화(Playing/Rebuffering), 실시간 그래프 렌더링 검증

04. 실험결과 및 성능분석 2분

발표 내용: 이론적 네트워크 수식(다운로드 시간, Throughput) 대비 실측 성능 데이터 비교 검증

핵심 내용: 네트워크 급변 시나리오별 ABR 알고리즘의 화질 유지력 및 버퍼 안정성 분석 결과 제시

05. 질의응답 (Q&A) 3분

대응 내용: 교수자 및 동료 평가자의 알고리즘 로직 및 통신 이론 관련 질문 답변

역할 분담: 팀원 전원 참여하여 모듈별 기술 질문에 맞춤형 답변 제공

2. 시연 리스크 관리 전략

백업 시연영상 사전 제작: 발표 당일 예상치 못한 장비 오류, 실행 환경 문제, 시스템 다운 등에 대비하여 OBS Studio를 활용한 3~5분 분량의 고화질 시연 영상을 사전 녹화 및 준비함.

로컬 독립 실행 환경 구축 (외부 의존성 최소화): 발표장의 네트워크 불안정성에 영향을 받지 않도록 외부 서버 연동 없이 로컬 환경에서 모든 데이터 파이프라인이 완결 구동되도록 시연 모드(--mode demo)를 구축함.

예외 처리 가드 및 오류 대응 매뉴얼 작성: UI 슬라이더 0 세팅 등 극단적인 입력값이나 패킷 손실 100% 상황에서도 프로그램이 다운되지 않는 예외 처리 구문을 구현하고, 비상시 대체 시연 순서가 담긴 대응 가이드를 수립함.

IV. 기타

개발 환경 표준화 및 의존성 세밀: 팀원 간 개발 및 실행 환경 통일을 위하여 Python 3.11 기반으로 프로젝트를 진행하며, requirements.txt에 명시된 필수 패키지 라이브러리를 한 번에 설치할 수 있도록 실행 매뉴얼(README.md)을 표준 폴더 구조(src/, docs/, results/)와 함께 정립함.

GitHub 커밋 이력 기반의 기여도 및 산출물 관리: 개인별 개발 기여도를 투명하게 증명하기 위해 기능별 Feature 브랜치 작업 후 PR(Pull Request) 검토를 거쳐 main 브랜치로 병합하는 체계를 유지하며, 최종 제출 산출물 5종(실행 프로그램, 소스코드 저장소, 실행 매뉴얼, 시연 시나리오 및 백업 영상, 최종 보고서)을 일괄 완비하여 정합성을 확보함.